

Exercício 1: Sistema de Mensagens (Stub e Spy Simplificado)

Contexto: Imagine que você tem um módulo simples responsável por enviar mensagens. Para testar a lógica que decide **quando e com o que** uma mensagem deve ser enviada, sem realmente disparar um e-mail ou SMS, usaremos **Stubs e Spies**.

Objetivo: Testar uma função que processa o envio de mensagens, garantindo que a função de envio subjacente seja chamada com os parâmetros corretos e que o retorno seja o esperado.

Estrutura do Projeto

```
exerciciol/  
├─ app.py  
└─ test_app.py
```

Tarefas

1. No arquivo **app.py**:

- Crie uma função `enviar_mensagem_externa(destinatario: str, conteudo: str) -> str`. Esta função deve apenas retornar uma string como `"Mensagem enviada para {destinatario}: {conteudo}"`. Pense nela como a dependência externa que não queremos chamar de verdade.
- Crie uma função `processar_envio(destinatario: str, texto: str) -> str`. Esta função deve chamar `enviar_mensagem_externa` e retornar o resultado.

2. No arquivo **test_app.py**:

- Utilize `pytest` e `mock` (do `pytest-mock`) para substituir a função `enviar_mensagem_externa` por um Stub/Spy.
- Crie um teste para verificar se `enviar_mensagem_externa` é chamada exatamente uma vez com os argumentos corretos (`destinatario` e `conteudo`).
- Configure o Stub para retornar um valor fixo e verifique se `processar_envio` retorna esse valor.

Dicas

- Use `mock.patch('app.enviar_mensagem_externa')` para substituir a função.
- Configure o retorno do seu mock com `mock_funcao.return_value = "..."`.

- Verifique as chamadas com `mock_funcao.assert_called_once_with(arg1, arg2)`.

Exercício 2: Gerenciador de Usuários (Fake Simplificado)

Contexto: Você está desenvolvendo um módulo simples para gerenciar usuários, onde cada usuário tem apenas um nome. Este módulo precisa interagir com um “repositório” para armazenar e recuperar os nomes dos usuários. Para testar a lógica do gerenciador sem a complexidade de um banco de dados real, você criará uma implementação **Fake** do repositório.

Objetivo: Testar as operações de adicionar e buscar usuários, utilizando um repositório em memória que simula o comportamento de um banco de dados real, mas de forma simplificada.

Estrutura do Projeto

```
exercicio2/  
├── app.py  
└── test_app.py
```

Tarefas

1. No arquivo **app.py**:

- Crie uma interface (ou classe abstrata) `IRepositorioUsuarios` com os métodos `adicionar_usuario(nome: str)` e `buscar_usuario(nome: str) -> Optional[str]`.
- Crie uma implementação **Fake** dessa interface, chamada `RepositorioUsuariosFake`. Esta classe deve usar uma lista ou um dicionário Python simples para armazenar os nomes dos usuários em memória.
- Crie uma classe `GerenciadorUsuarios` que recebe uma instância de `IRepositorioUsuarios` no seu construtor. Esta classe terá métodos como `registrar_usuario(nome: str)` e `encontrar_usuario(nome: str)` que utilizam o repositório.

2. No arquivo **test_app.py**:

- Utilize `pytest` para testar a classe `GerenciadorUsuarios`.
- Nos testes, injete uma instância de `RepositorioUsuariosFake` no `GerenciadorUsuarios`.
- Crie um teste para adicionar um usuário e verificar se ele pode ser encontrado posteriormente.

- Crie um teste para tentar buscar um usuário que não foi adicionado e verificar se o retorno é `None`.

Dicas

- A implementação Fake deve ser o mais simples possível, focando apenas em simular o armazenamento e recuperação de nomes.
- Use fixtures do `pytest` para instanciar o `RepositorioUsuariosFake` e o `GerenciadorUsuarios` em cada teste, garantindo isolamento.

Exercício 3: Processador de Ações (Mock Simplificado)

Contexto: Você tem um processador de ações que precisa executar duas etapas em uma ordem muito específica: primeiro, `validar_dados` e, em seguida, `salvar_dados`. A ordem é crucial para evitar salvar dados inválidos. Para garantir que essa sequência de chamadas seja rigorosamente seguida, você utilizará **Mocks**.

Objetivo: Testar uma função `executar_processo` para garantir que as funções `validar_dados` e `salvar_dados` sejam chamadas na ordem correta e com os parâmetros esperados.

Estrutura do Projeto

```
exercicio3/  
├── app.py  
└── test_app.py
```

Tarefas

1. No arquivo `app.py`:

- Crie uma classe `ProcessadorDeAcoes`.
- Dentro dela, crie dois métodos: `validar_dados(dados: dict) -> bool` e `salvar_dados(dados: dict) -> bool`. Ambos devem apenas simular a operação, retornando `True`.
- Crie um método `executar_processo(dados: dict)` que chama `validar_dados` e, se for bem-sucedido, chama `salvar_dados`. Ele deve retornar uma mensagem de sucesso ou falha.

2. No arquivo `test_app.py`:

- Utilize `pytest` e `mock` (do `pytest-mock`) para criar um Mock da classe `ProcessadorDeAcoes`.
- Crie um teste que chame `executar_processo` com o Mock.

- Use `mock_objeto.assert_has_calls(roteiro_esperado, any_order=False)` para verificar a ordem exata das chamadas e os parâmetros passados para os métodos do Mock.
- Verifique se a função `executar_processo` retorna a mensagem de sucesso esperada.

Dicas

- Lembre-se de importar `call` de `unittest.mock` para construir o `roteiro_esperado`.
- O foco principal deste exercício é a **verificação da ordem das interações** e dos **parâmetros** passados, que é a principal força dos Mocks.